

Classwork: Building the EPQ and ABC Classification Models

How to use this classwork

This follows exactly the same approach as the EOQ tutorial: read a short explanation, see the formula, then build it in Python one step at a time

Part 1 — Economic Production Quantity (EPQ)

1. What Is EPQ, and Why Does It Matter?

EPQ is a close relative of EOQ, used when you manufacture your own stock rather than ordering it from a supplier. Because production happens gradually, you're producing and consuming stock at the same time — inventory never actually piles up to the full batch size the way it does with EOQ. That changes the holding cost side of the calculation, which is the only real difference between the two models.

2. The EPQ Formula

The formula looks just like EOQ, with one extra factor in the denominator:

$$EPQ = \sqrt{\frac{2DS}{H \left(1 - \frac{d}{p}\right)}}$$

Where:

- D = annual demand (units/year)
- S = setup cost per production run (a fixed cost, in currency)
- H = holding cost per unit per year (in currency)
- d = daily demand rate (units/day)
- p = daily production rate (units/day)

Two other values usually come alongside it:

- Number of production runs per year = D / EPQ
- Maximum inventory level reached = $EPQ \times (1 - d/p)$ — because production and consumption happen together, inventory never reaches the full EPQ amount

3. Building It in Python, Step by Step

Step 1 Import what you need

```
import math
```

Step 2 Define your inputs

```
annual_demand = 12000      # units per year
setup_cost = 50             # cost per production run, in Rand
holding_cost = 2            # cost per unit per year, in Rand
daily_demand_rate = 40      # units produced/sold per day
daily_production_rate = 100  # units your process can make per day
```

Step 3 Write the EPQ function

```
def calculate_epq(demand, setup, hold_cost, d_rate, p_rate):
    return math.sqrt((2 * demand * setup) / (hold_cost * (1 - d_rate / p_rate)))

epq = calculate_epq(annual_demand, setup_cost, holding_cost,
                    daily_demand_rate, daily_production_rate)
print("Optimal production quantity:", round(epq, 2))
```

Step 4 Calculate production runs per year and run length

```
runs_per_year = annual_demand / epq
run_length_days = epq / daily_production_rate

print("Production runs per year:", round(runs_per_year, 2))
print("Length of each run (days):", round(run_length_days, 1))
```

Step 5 Calculate maximum inventory level

Because you're consuming stock while producing it, inventory never actually reaches the full EPQ — it peaks lower than that.

```
max_inventory = epq * (1 - daily_demand_rate / daily_production_rate)
print("Maximum inventory level:", round(max_inventory, 2))
```

Step 6 Put it all together

```
import math

def calculate_epq(demand, setup, hold_cost, d_rate, p_rate):
    return math.sqrt((2 * demand * setup) / (hold_cost * (1 - d_rate / p_rate)))

annual_demand = 12000
setup_cost = 50
holding_cost = 2
daily_demand_rate = 40
daily_production_rate = 100
```

```

epq = calculate_epq(annual_demand, setup_cost, holding_cost,
                   daily_demand_rate, daily_production_rate)
runs_per_year = annual_demand / epq
run_length_days = epq / daily_production_rate
max_inventory = epq * (1 - daily_demand_rate / daily_production_rate)

print("Optimal production quantity:", round(epq, 2))
print("Production runs per year:", round(runs_per_year, 2))
print("Length of each run (days):", round(run_length_days, 1))
print("Maximum inventory level:", round(max_inventory, 2))

```

4. Worked Example

With $D = 12,000$, $S = R50$, $H = R2$, daily demand = 40, daily production = 100, the script produces:

```

Optimal production quantity: 1000.0
Production runs per year: 12.0
Length of each run (days): 10.0
Maximum inventory level: 600.0

```

Interpretation: rather than one huge batch, the model recommends 12 production runs a year, each producing 1,000 units over 10 days — and because you're selling stock while you make it, inventory never actually piles higher than 600 units at once.

5. Try It Yourself

1. Change `daily_production_rate` to 150 (a faster machine). What happens to the EPQ — does it go up or down? Why does that make sense?
2. Notice that if `daily_production_rate` is very large compared to demand, EPQ approaches plain EOQ. Try $p = 100,000$ and compare the result to the plain EOQ formula from the tutorial (with the same D , S , H) — they should be nearly identical. Can you explain why?

Part 2 — ABC Inventory Classification

1. What Is ABC Classification, and Why Does It Matter?

Not every SKU deserves the same attention. ABC classification ranks your inventory items by how much they contribute to total inventory value, so you can focus tight controls — frequent counts, safety stock, supplier negotiation — on the vital few, rather than treating every SKU identically.

It's based on the Pareto principle: a small share of SKUs (Tier A) typically accounts for most of the value, while a large share of SKUs (Tier C) contributes very little.

Unlike EOQ and EPQ, this isn't a single-number formula — it's a small data-processing pipeline: calculate a value per item, sort, accumulate, then classify.

2. The Approach

For each SKU, calculate its annual usage value:

$$UsageValue = D \times C$$

Where D = annual demand for that SKU, and C = its unit cost.

Then, across all SKUs sorted from highest usage value to lowest, track the running (cumulative) share of total value:

$$CumulativePct = \frac{RunningTotal}{TotalValue} \times 100$$

Finally, classify each SKU based on where its cumulative percentage falls:

- Tier A — cumulative value up to roughly 80%
- Tier B — cumulative value from roughly 80% to 95%
- Tier C — the remaining ~5%, usually the majority of SKUs by count

3. Building It in Python, Step by Step

Step 1 — Represent your SKU data

This is exactly the list-of-dictionaries pattern from Module 03 — one dictionary per SKU.

```
skus = [  
    {"sku": "BRK-100", "demand": 2000, "cost": 45},  
    {"sku": "GSK-220", "demand": 1500, "cost": 30},  
    {"sku": "BLT-010", "demand": 10000, "cost": 2},  
    {"sku": "BRG-330", "demand": 800, "cost": 60},  
    {"sku": "SEAL-500", "demand": 3000, "cost": 5},  
    {"sku": "MTR-700", "demand": 50, "cost": 800},  
    {"sku": "WSH-050", "demand": 20000, "cost": 0.5},  
    {"sku": "CBL-900", "demand": 400, "cost": 25},  
]
```

Step 2 Calculate usage value per SKU

```
def usage_value(demand, cost):
    return demand * cost

for item in skus:
    item["value"] = usage_value(item["demand"], item["cost"])
```

Step 3 Sort descending by value

```
skus_sorted = sorted(skus, key=lambda item: item["value"], reverse=True)
```

Step 4 Calculate cumulative percentage

```
total_value = sum(item["value"] for item in skus_sorted)

running_total = 0
for item in skus_sorted:
    running_total += item["value"]
    item["cum_pct"] = (running_total / total_value) * 100
```

Step 5 Assign a tier

```
def assign_tier(cum_pct):
    if cum_pct <= 80:
        return "A"
    elif cum_pct <= 95:
        return "B"
    else:
        return "C"

for item in skus_sorted:
    item["tier"] = assign_tier(item["cum_pct"])
```

Step 6 Print the classification report

```
for item in skus_sorted:
    print(item["sku"], "| value:", item["value"],
          "| cum %:", round(item["cum_pct"], 1),
          "| tier:", item["tier"])
```

Step 7 Count SKUs per tier

A dictionary makes a natural running counter — exactly like the pattern from Module 03.

```
tier_counts = {"A": 0, "B": 0, "C": 0}
for item in skus_sorted:
    tier_counts[item["tier"]] += 1

print(tier_counts)
```

4. Worked Example

Running the full script on the 8 SKUs above produces:

```
BRK-100 | value: 90000 | cum %: 32.4 | tier: A
BRG-330 | value: 48000 | cum %: 49.6 | tier: A
GSK-220 | value: 45000 | cum %: 65.8 | tier: A
MTR-700 | value: 40000 | cum %: 80.2 | tier: B
BLT-010 | value: 20000 | cum %: 87.4 | tier: B
SEAL-500 | value: 15000 | cum %: 92.8 | tier: B
WSH-050 | value: 10000 | cum %: 96.4 | tier: C
CBL-900 | value: 10000 | cum %: 100.0 | tier: C

{'A': 3, 'B': 3, 'C': 2}
```

Interpretation: just 3 of the 8 SKUs (BRK-100, BRG-330, GSK-220) account for nearly two-thirds of total inventory value. Those are the items worth counting more often, negotiating harder on, and never letting go out of stock — while the Tier C items can be managed with far less attention.

5. Try It Yourself

1. Add two more SKUs of your own invention to the list. Does the tier split change much?
2. Change the classification thresholds to 70% / 90% instead of 80% / 95%. How do the tier counts change?
3. Wrap Steps 2–6 into a single function called `classify_inventory(skus)` that returns the sorted, classified list — exactly the kind of function you built for EOQ.

What to Submit

- Two Python files: `yourname_epq.py` and `yourname_abc.py` (or one combined file with clearly labeled sections — check with your instructor which is preferred)
- Both scripts should run without errors and print clearly labeled results
- A one- or two-sentence comment at the top of each file explaining what the model does, in your own words
- Submit at the end of class through the usual submission channel